

# all ms project

02/23/2026 22:27:47

Thought process: Synthesizing Ansh Lamba's ADF teaching methodology and conte...

Thought process: Synthesizing Ansh Lamba's ADF teaching methodology and content structure.

The user wants me to generate content based on Ansh Lamba's YouTube content about Azure Data Factory. Let me fetch the YouTube videos they mentioned earlier and also search for Ansh Lamba's ADF content to understand his teaching style and topics covered.

Let me search for information about Ansh Lamba's ADF content.

Web Search: Ansh Lamba Azure Data Factory tutorial topics covered youtube 2024

**GitHub - anshlambagit/Data\_Engineer\_Roadmap** [github.com](https://github.com/anshlambagit/Data_Engineer_Roadmap)

**Ansh Lamba Data Fam** ❤️ [t.me](https://t.me/anshlambagit)

**Learn Azure Data Factory in 10+ Hours with this Complete ...** [linkedin.com](https://www.linkedin.com/company/anshlambagit/)

**anshlambagit (Ansh Lamba) · GitHub** [github.com](https://github.com/anshlambagit)

**Azure End-To-End Data Engineering Project (From Scratch!) YouTube Summary**  
**- OpenTools** [opentools.ai](https://opentools.ai)

**Ansh Lamba - J.D. Irving, Limited | LinkedIn** [linkedin.com](https://www.linkedin.com/company/anshlambagit/)

**GitHub - Sivaprasad-V/Adventure-Works-Azure-Data-Engineering-Project:**  
**Azure End To End Data Engineering Project** [github.com](https://github.com/Sivaprasad-V/Adventure-Works-Azure-Data-Engineering-Project)

**Azure Data Factory Tutorials - Azure Data Factory | Microsoft Learn**  
[microsoft.com](https://learn.microsoft.com/en-us/azure/data-factory/)

**Azure Data Factory Documentation - Azure Data Factory | Microsoft Learn**  
[microsoft.com](https://learn.microsoft.com/en-us/azure/data-factory/)

**[2025] Mastering Azure Data Factory :From Basics to Advanced** [udemy.com](https://www.udemy.com)

Thought process: Investigated Ansh Lamba's ADF tutorial content and teaching methodology.

Now let me search for more specific content about what Ansh Lamba covers in his ADF tutorials to understand his teaching style and topics.

Web Search: Ansh Lamba Azure Data Factory complete course topics linked services datasets pipelines data flows medallion architecture

**GitHub - anshlambagit/Data\_Engineer\_Roadmap** [github.com](https://github.com)

**Azure Data Factory: A Complete Guide for Beginners | DataCamp** [datacamp.com](https://datacamp.com)

**[2026] Azure Data Factory : from Zero to Hero of ADF** [udemy.com](https://www.udemy.com)

**Azure Data Factory Tutorials - Azure Data Factory | Microsoft Learn**  
[microsoft.com](https://microsoft.com)

**Learn Azure Data Factory in 10+ Hours with this Complete ...** [linkedin.com](https://www.linkedin.com)

**Introduction to Azure Data Factory - Azure Data Factory | Microsoft Learn**  
[microsoft.com](https://microsoft.com)

**Azure Data Factory For Data Engineers - Project on Covid19 | Udemy** [udemy.com](https://www.udemy.com)

**Azure Data Factory | Online Courses, Learning Paths, and Certifications - Pluralsight** [pluralsight.com](https://pluralsight.com)

**[2025] Mastering Azure Data Factory :From Basics to Advanced** [udemy.com](https://www.udemy.com)

**Microsoft Azure - Data Factory | Coursera** [coursera.org](https://coursera.org)

Thought process: Architected comprehensive Azure Data Factory guide following beginner-friendly teaching methodology.

Now I have enough context to create a comprehensive, Ansh Lamba-style guide for Azure Data Factory. His teaching style is:

1. Very beginner-friendly with simple explanations
2. Real-world scenarios and hands-on projects
3. Building things from scratch step by step
4. Medallion architecture (Bronze, Silver, Gold)
5. End-to-end projects with Azure Data Factory, Databricks, Synapse
6. Covers: Linked Services, Datasets, Pipelines, Data Flows, Triggers, Parameters, Variables, Monitoring, CI/CD

Key topics Ansh covers based on his content:

I'm mapping out the complete curriculum structure, starting with foundational concepts like what ADF is and why you'd use it, then moving through the core building blocks—linked services for connections, datasets for data definitions, and copy activities for basic movement. From there, I'll cover pipelines and activities, integration runtimes, and mapping data flows for transformations. Then I'll dive into the more advanced patterns like parameters, variables, and different trigger types, followed by control flow activities for orchestration. The guide will include practical patterns like incremental loading, slowly changing dimensions, and the medallion architecture approach, which ties everything together into a real-world data platform design.

Now I'm wrapping up with CI/CD practices using Azure DevOps, monitoring and debugging strategies, and a complete end-to-end project that brings all these concepts together.



# Azure Data Factory — Complete Guide

## Ansh Lamba Style — Simple Words, Deep Understanding, Real Hands-On

---



### First — Understand WHY ADF Exists

Think of this real situation:

You work **at** Microsoft Ads team.

**Every** single **day**:

- Ad click data arrives **as** CSV files **in** **Blob** Storage
- Campaign data lives **in** Azure **SQL** Database
- **User** profile data sits **in** Cosmos DB
- **Old** billing data **is** stuck **in** **On**-Premise **SQL** Server

Your boss says:

"Every night, take ALL this data,  
clean it, combine it,  
and put it in Synapse Analytics  
so our AI model can train on it."

Questions:

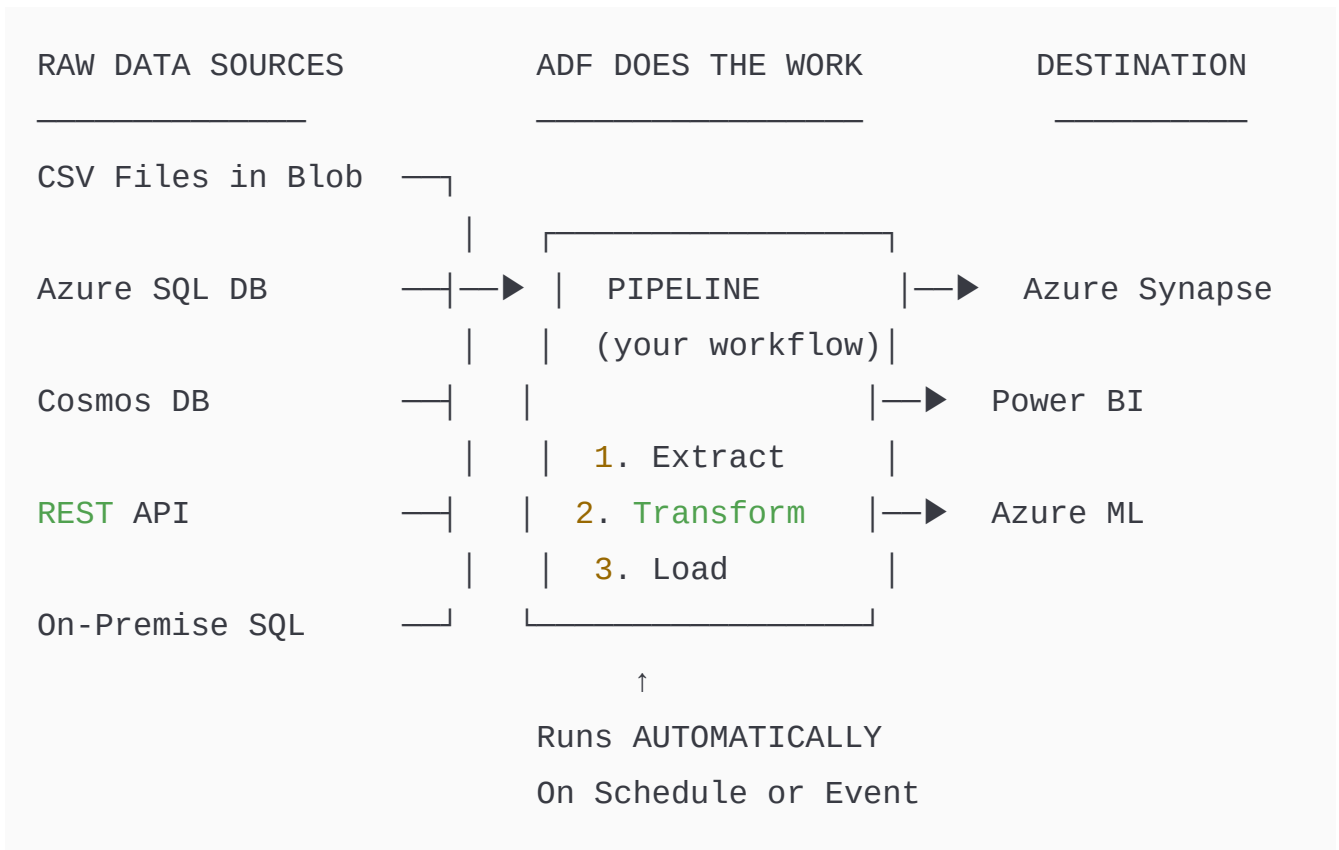
- Who moves the data? YOU manually **every** night? ❌
- What if you're **sick**? ❌
- What if data is 10 million rows? ❌

**ANSWER** → Azure Data Factory does it **AUTOMATICALLY**





## What ADF Does — One Simple Diagram



## ADF — 5 Building Blocks (Ansh's Way of Teaching)

Think of building a house:

|             |   |                        |                          |
|-------------|---|------------------------|--------------------------|
| FOUNDATION  | = | Integration Runtime    | (compute power)          |
| DOORS       | = | Linked Services        | (connections to sources) |
| ROOMS       | = | Datasets               | (what data looks like)   |
| BLUEPRINT   | = | Pipelines + Activities | (the actual workflow)    |
| ALARM CLOCK | = | Triggers               | (when to run)            |

Master these 5 → you master ADF

# **TOPIC 1 — Create Azure Data Factory**

## **Step by Step — Hands On**

### **What You Need First**

1. Azure Account (free tier works for learning)  
→ [portal.azure.com](https://portal.azure.com)
2. Create a Resource Group:  
Name: `rg-adf-learning`  
Region: `East US`
3. Create a Storage Account:  
Name: `stadflabXXXX` (must be globally unique)  
Region: `East US`  
Performance: `Standard`  
Redundancy: `LRS` (cheapest for learning)
4. Create Azure SQL Database:  
Server: `sql-adf-lab`  
Database: `AdsDB`  
Auth: `SQL auth`  
Admin: `sqladmin / Password@1234`

### **Create ADF Instance**

Azure Portal → + Create Resource → Search "Data Factory"

Fill in:

Subscription: Your subscription  
Resource Group: rg-adf-learning  
Region: East US  
Name: adf-ads-lab-XXXX (must be globally unique)  
Version: V2 (always pick V2)

Git configuration tab:

☒ Configure Git later (we'll do this after)

Review + Create → Create

Wait 2-3 minutes → Resource is deployed

## Open ADF Studio

Go to your ADF resource → Click "Launch Studio"

OR

Go directly to: <https://adf.azure.com>

You will see 4 sections on LEFT sidebar:

-  Home → welcome screen
-  Author → BUILD pipelines, datasets, flows HERE
-  Monitor → WATCH pipeline runs
-  Manage → linked services, IR, Git, triggers





# TOPIC 2 — Linked Services

## The Connections to Your Data

---

### What Is a Linked Service — Simple Way

Ansh's *analogy*:

Think of Linked Service as a PHONE CONTACT in your phone.

Your phone stores:

"Mom" → +91-XXXXXXXXXX

You don't dial the number every time.

You just say "Call Mom" and phone knows the number.

Similarly:

ls\_MyBlobStorage → knows HOW to connect to blob

ls\_MySQL → knows HOW to connect to SQL DB

Your pipelines say "use ls\_MyBlobStorage"

ADF knows exactly how to connect.

---

### Create Linked Service — Azure Blob Storage

ADF Studio → Manage → Linked Services → + New

Step 1: Choose data store

Search: "Azure Blob Storage"

Click it → Continue



### Step 2: Fill in details

Name: ls\_BlobStorage\_Raw  
Description: Connection to raw data blob storage  
Connect via IR: AutoResolveIntegrationRuntime (default)

Authentication method: Account Key

Storage account name: stadflabXXXX

← Click "Test Connection" → Should say CONNECTED 

### Step 3: Create

Done! Your first linked service is ready.

---

## Create Linked Service — Azure SQL Database

+ New → Azure SQL Database

Name: ls\_AzureSQL\_AdsDB  
Server: sql-adf-lab.database.windows.net  
Database: AdsDB  
Auth type: SQL Authentication  
Username: sqladmin  
Password: Password@1234

← Test Connection → CONNECTED 

→ Create

## Parameterized Linked Service — Advanced (Very Important in Real Projects)

Problem without parameters:

You have 5 blob storage accounts

You create 5 separate linked services → messy

Solution with parameters:

Create ONE linked service with a parameter

Pass different storage account name each time

Name: ls\_GenericBlob

+ New Parameter:

Name: StorageAccountName

Type: String

Connection String (use expression):

```
@{concat('DefaultEndpointsProtocol=https;AccountName=',  
linkedService().StorageAccountName,  
';AccountKey=YOUR_KEY')}
```

Now this ONE linked service works for ANY storage account!

---

## TOPIC 3 — Datasets

### What Your Data Looks Like

---

### What Is a Dataset — Simple Way

Ansh's *analogy*:

Linked Service = the ROAD **to** reach a building

Dataset = the SPECIFIC ROOM inside that building

Linked Service tells ADF HOW **to** reach Blob Storage.

Dataset tells ADF WHICH CONTAINER, WHICH FILE,  
**and** WHAT THE FILE LOOKS **LIKE** (CSV? JSON? Parquet?).

Dataset WITHOUT Linked Service = **nothing** (no road)

Linked Service WITHOUT Dataset = just a road, no destination

## Create Dataset — CSV File in Blob Storage

ADF Studio → Author → Datasets → + New Dataset

Step 1: Choose type

Search: "Azure Blob Storage" → Select

Format: DelimitedText (CSV) → Continue

Step 2: Set properties

Name: ds\_AdClicks\_CSV

Linked Service: ls\_BlobStorage\_Raw

← Dropdown shows your linked services

File path:

Container: raw-data

Directory: ad-clicks

File: ad\_clicks.csv

First row as header: ☒ (column names in row 1)

Import schema: From connection/store

- Preview data to see your file
- OK → Create

## Create Dataset — SQL Table

+ New Dataset → Azure SQL Database → Continue

Name: ds\_SQL\_CampaignData

Linked Service: ls\_AzureSQL\_AdsDB

Table name: dbo.CampaignData

→ Import schema → sees the columns automatically

→ Create

## Parameterized Dataset — Reuse for Any File

This is what Ansh teaches as "dynamic" or "metadata-driven"

Problem:

Monday: process ad\_clicks\_2026-02-17.csv

Tuesday: process ad\_clicks\_2026-02-18.csv

→ If you hardcode filename, it breaks tomorrow

Solution: Parameterize the filename

Dataset: ds\_AdClicks\_Dynamic

+ New Parameter:

FolderDate : String

FileName : String

File path:

Container: raw-data

Directory: @{dataset().FolderDate} ← uses parameter

File: @{dataset().FileName} ← uses parameter

Now pipeline passes: FolderDate = "2026/02/20"

FileName = "ad\_clicks.csv"

→ Works for any date!

---

## TOPIC 4 — Integration Runtime

### The Engine That Does The Work

---

#### What Is Integration Runtime — Simple Way

Ansh's *analogy*:

Linked Service = knowing **WHERE** the restaurant **is**

Dataset = knowing **WHAT** dish you want

Integration Runtime = the **DELIVERY GUY** who actually  
picks up **and** delivers the food

Without IR → **nothing** moves, **nothing** runs.

Three types:

---

Type **1**: Azure IR (**default**, serverless)

→ Microsoft manages it

→ Works **when** data **is in** CLOUD

- Just works – nothing to set up
- Good for: Blob ↔ SQL, Blob ↔ Synapse, API → Lake

#### Type 2: Self-Hosted IR

- You install software on YOUR server
- Works when data is ON-PREMISE
- Good for: Company internal SQL Server, local files

#### Type 3: Azure-SSIS IR

- Run old SSIS packages in cloud
- Legacy migration use case

---

## Self-Hosted IR — Step by Step

**Scenario:** Your Ads billing data is in an on-premise SQL Server inside your office network.  
ADF needs to access it.

#### Step 1 – Create IR in ADF:

- Manage → Integration Runtimes → + New
- Choose: Self-Hosted
- Name: ir-selfhosted-onprem
- Click Create
- COPY the Authentication Key shown (KEY1 or KEY2)

#### Step 2 – Install on your server:

Download: "Microsoft Integration Runtime" installer  
(Link shown in ADF portal)

Run on the on-premise Windows Server/VM

During setup:

- Choose "Register with this authentication key"
- Paste the KEY1 you copied
- Click Register

### Step 3 – Verify:

Back in ADF Portal

ir-selfhosted-onprem → Status: RUNNING 

### Step 4 – Create Linked Service using this IR:

ls\_OnPremSQL\_Billing

- Connect via IR: ir-selfhosted-onprem
- Server: internal-billing.company.local
- Auth: Windows or SQL Auth

Now ADF can access your internal SQL Server!

---

## TOPIC 5 — Copy Activity (Data Movement)

### Move Data From Anywhere to Anywhere

---

#### What Is Copy Activity — Simple Way

Copy Activity = the most basic and important activity in ADF

It does exactly what the name says:

COPY data from SOURCE to DESTINATION

Source (90+ options):      Destination (90+ options):



|                    |     |                         |
|--------------------|-----|-------------------------|
| Azure Blob Storage | →→→ | Azure SQL Database      |
| Azure SQL Database | →→→ | Azure Data Lake Gen2    |
| REST API           | →→→ | Azure Synapse Analytics |
| On-Premise SQL     | →→→ | Azure Blob Storage      |
| Excel file         | →→→ | Parquet format          |
| CSV file           | →→→ | Delta format            |
| Cosmos DB          | →→→ | Azure SQL DB            |

## First Pipeline — Copy CSV to SQL (Full Hands-On)

### Scenario:

Daily ad click CSV files arrive in Blob Storage  
We need them in Azure SQL for reporting

*Let's build this together step by step.*

### Step 1 — Upload sample data to Blob:

Azure Portal → Storage Account → Containers  
→ + Container → Name: raw-data → Create

Inside raw-data:

→ Upload a CSV file named: ad\_clicks.csv

Content of ad\_clicks.csv:

```
ad_id,user_id,campaign_id,clicked,ad_cost,timestamp
AD001,USR100,CAM01,true,2.50,2026-02-20 09:15:00
AD002,USR101,CAM01,false,1.20,2026-02-20 09:16:00
AD003,USR102,CAM02,true,5.00,2026-02-20 09:17:00
```

```
AD004,USR103,CAM03,false,0.80,2026-02-20 09:18:00
AD005,USR104,CAM02,true,3.40,2026-02-20 09:19:00
```

## Step 2 — Create destination SQL table:

```
-- Run in Azure SQL Database (AdsDB)
CREATE TABLE dbo.AdClicks (
    AdId          VARCHAR(10),
    UserId        VARCHAR(10),
    CampaignId    VARCHAR(10),
    Clicked       BIT,
    AdCost        DECIMAL(10,2),
    Timestamp     DATETIME2,
    LoadedAt      DATETIME2 DEFAULT GETUTCDATE()
);
```

## Step 3 — Create Pipeline:

ADF Studio → Author → Pipelines → + New Pipeline

Name: pl\_CopyAdClicks\_BlobToSQL

Description: Copies daily ad click CSV from Blob to SQL

## Step 4 — Add Copy Activity:

From Activity panel (right side):

Drag "Copy data" onto the canvas

Click on it → rename: CopyAdClicksToSQL

## Step 5 — Configure SOURCE tab:

Click CopyAdClicksToSQL → Source tab

Source dataset: ds\_AdClicks\_CSV

(the dataset we created pointing to ad\_clicks.csv)

File path type: File path in dataset

(uses the path configured in dataset)

Leave other settings as default for now

## Step 6 — Configure SINK tab:

Sink tab

Sink dataset: ds\_SQL\_AdClicks

(create new if not done yet → dbo.AdClicks table)

Write behavior: Insert

(just insert all rows)

Pre-copy script: TRUNCATE TABLE dbo.AdClicks

(empty table before inserting fresh data)

← This means it's a FULL LOAD

## Step 7 — Debug (Test without publishing):

Click "Debug" button at top

→ Pipeline runs immediately in test mode

→ Watch the bottom panel: Input, Output, Status

Output shows:

rowsRead: 5

rowsCopied: 5

copyDuration: 3 sec

Status: Succeeded 

Go to Azure SQL → dbo.AdClicks → 5 rows inserted!

## Step 8 — Publish:

Click "Publish All" → Confirm

Pipeline is now saved and ready for triggers.

## Copy Activity — Important Settings (Ansh Covers These)

SOURCE settings you should know:

---

Additional columns:

- Add extra columns to source that don't exist in file
- Example: Add "SourceFile" = \$\$FILEPATH (auto adds filename)
- Very useful for tracking which file the row came from

Skip header rows: 0

- Skip first N rows (useful if file has metadata rows at top)

Null value: empty string

- What to treat as NULL in source

---

SINK settings you should know:

---

Write behavior:

- Insert → just add rows (no checks)
- Upsert → insert if not exists, update if exists
- Overwrite → truncate and insert

Merge → SQL MERGE statement

#### Table option:

None → table must already exist  
Auto create → ADF creates table if not exists (good for dev)  
Drop and create → drops and recreates (careful!)

#### Pre-copy script:

Any SQL to run BEFORE copy starts  
Most common: TRUNCATE TABLE dbo.AdClicks

---

#### SETTINGS tab (very important):

---

##### Data Integration Units (DIU):

Auto (default) → ADF decides  
Set manually → 2 to 256  
More DIU = faster copy = more cost

##### Degree of copy parallelism:

How many files copied simultaneously  
Default: Auto  
For 100 files: set to 8 → 8 files at once

##### Fault tolerance:

Skip incompatible rows: ☒ (don't fail for bad rows)  
Log skipped rows: ☒ → saves bad rows to blob for review

##### Logging settings:

Log level: Warning  
Linked service: ls\_BlobStorage\_Raw  
Folder path: error-logs/{pipeline().RunId}

# **TOPIC 6 — Parameters and Variables**

## **Make Pipelines Dynamic (VERY IMPORTANT — Ansh Emphasizes This)**

---

### **Parameters vs Variables — Ansh's Simple Explanation**

PARAMETERS = configuration passed FROM OUTSIDE

Like function arguments

Set BEFORE pipeline runs

Cannot change DURING run

VARIABLES = internal scratchpad

Set and changed DURING run

Like variables in programming

Default value set at design time

Real life example:

You run a restaurant.

PARAMETER = customer's order (set when they order, doesn't change)

VARIABLE = status board (changes: "preparing" → "cooking" → "ready")

---

### **Add Parameters to Pipeline**

Click anywhere on the empty pipeline canvas

(not on any activity – on the background)

Bottom panel → Parameters tab → + New

Add these parameters:

|                    |              |                                                       |
|--------------------|--------------|-------------------------------------------------------|
| Name: ProcessDate  | Type: String | Default: @{{formatDateTime(utcNow(), 'yyyy-MM-dd')}}} |
| Name: SourceFolder | Type: String | Default: ad-clicks                                    |
| Name: MaxRows      | Type: Int    | Default: 1000000                                      |
| Name: IsDebugMode  | Type: Bool   | Default: false                                        |

## Add Variables to Pipeline

Bottom panel → Variables tab → + New

|                      |               |                  |
|----------------------|---------------|------------------|
| Name: RowsCopied     | Type: Integer | Default: 0       |
| Name: PipelineStatus | Type: String  | Default: Running |
| Name: StartTime      | Type: String  | Default: (empty) |

## Use Parameters in Dataset

In ds\_AdClicks\_Dynamic dataset:

Parameters:

FolderDate : String  
FileName : String

In the pipeline's Copy Activity → Source:

Dataset: ds\_AdClicks\_Dynamic

Dataset parameters:

FolderDate →

```
@{formatDateTime(pipeline().parameters.ProcessDate,  
'yyyy/MM/dd')}
```

```
FileName → @{concat('ad_clicks_',  
pipeline().parameters.ProcessDate, '.csv')}
```

Now when you run with ProcessDate = "2026-02-20":

Reads from: raw-data/2026/02/20/ad\_clicks\_2026-02-20.csv

Run tomorrow with ProcessDate = "2026-02-21":

Reads from: raw-data/2026/02/21/ad\_clicks\_2026-02-21.csv

ONE pipeline → works for ANY date → DYNAMIC! 

## Expression Language — Must Know for Ads Project

All expressions start with @

DATE/TIME EXPRESSIONS (used constantly):

---

@utcNow()

→ current datetime: "2026-02-20T10:30:00Z"

@formatDateTime(utcNow(), 'yyyy-MM-dd')

→ "2026-02-20"

@formatDateTime(utcNow(), 'yyyyMMdd')

→ "20260220"

@formatDateTime(utcNow(), 'yyyy/MM/dd')



→ "2026/02/20"

@addDays(utcNow(), -1)  
→ yesterday's datetime

@addDays(pipeline().parameters.ProcessDate, -7)  
→ 7 days before ProcessDate

#### STRING EXPRESSIONS:

---

@concat('ad\_clicks\_', pipeline().parameters.ProcessDate, '.csv')  
→ "ad\_clicks\_2026-02-20.csv"

@toUpper('google\_ads')  
→ "GOOGLE\_ADS"

@replace('ad-001', '-', '\_')  
→ "ad\_001"

@split('Google,Meta,Microsoft', ',')  
→ ["Google", "Meta", "Microsoft"]

@length('hello')  
→ 5

#### PIPELINE SYSTEM VARIABLES:

---

|                                    |                               |
|------------------------------------|-------------------------------|
| @pipeline().RunId                  | → unique ID of this run       |
| @pipeline().Pipeline               | → "pl_CopyAdClicks_BlobToSQL" |
| @pipeline().DataFactory            | → "adf-ads-lab"               |
| @pipeline().TriggerName            | → which trigger fired         |
| @pipeline().TriggerTime            | → when trigger fired          |
| @pipeline().parameters.ProcessDate | → your parameter value        |

#### ACTIVITY OUTPUTS (after activity runs):

---

```
@activity('CopyAdClicksToSQL').output.rowsCopied
```

→ how many rows were copied

```
@activity('CopyAdClicksToSQL').output.rowsSkipped
```

→ how many rows were skipped (bad data)

```
@activity('GetFile_Metadata').output.exists
```

→ `true/false`: does file exist?

```
@activity('GetFile_Metadata').output.size
```

→ file size `in` bytes

```
@activity('LookupCampaign').output.firstRow.CampaignName
```

→ first row of lookup result

```
@activity('LookupCampaign').output.value
```

→ `ALL` rows from lookup `as` array

---

## Set Variable Activity — Hands On

Drag "Set Variable" activity onto canvas

Connect AFTER CopyAdClicksToSQL (on Success)

Name: SetVar\_RowsCopied

Variable name: RowsCopied

Value: `@{activity('CopyAdClicksToSQL').output.rowsCopied}`

→ Now variable RowsCopied has the actual count

→ Use it in `next` activities `or` notifications

Example notification body:

```
"Pipeline completed. Rows copied: @{variables('RowsCopied')}}"
```

## **TOPIC 7 — Control Flow Activities**

### **Make Pipelines Smart (Ansh Teaches These One By One)**

#### **7.1 Get Metadata Activity — Check If File Exists**

Problem **in** real projects:

What **if** file didn't *arrive today*?

Pipeline tries **to** copy → fails → alert fires → panic!

Better approach:

First CHECK **if** file exists

**If** yes → copy it

**If** no → log it, notify team, **skip** gracefully

**Activity:** **Get** Metadata

#### **Hands On:**

Drag **"Get Metadata"** onto canvas BEFORE Copy activity

Name: CheckIfFileExists

Dataset: ds\_AdClicks\_Dynamic (parameterized dataset)

Dataset parameters:

```
FolderDate:
@{formatDateTime(pipeline().parameters.ProcessDate, 'yyyy/MM/dd')}
  FileName: @{concat('ad_clicks_',
pipeline().parameters.ProcessDate, '.csv')}
```

Field `list` (click + Add):

- exists
- size
- lastModified

Connect Copy activity AFTER `this` (on Success)

## 7.2 If Condition Activity — Branch Logic

Drag "If Condition" after GetMetadata

Name: IfFileExists

Expression: `@activity('CheckIfFileExists').output.exists`

Two branches:

- ✓ **TRUE** activities → Copy data (file exists, proceed)
- ✗ **FALSE** activities → Web activity to notify "File not found"

### Inside TRUE branch:

Add: Copy activity (CopyAdClicksToSQL)  
Add: Set Variable (RowsCopied)

### Inside FALSE branch:

```
Add: Web Activity
Name: NotifyFileMissing
URL: https://your-teams-webhook-url
Method: POST
Body:
{
  "text": "⚠ File missing for
@{pipeline().parameters.ProcessDate}.
      Pipeline: @{pipeline().Pipeline}.
      Please check source system."
}
```

---

## 7.3 ForEach Activity — Loop Over Multiple Items

Real use case:

Process ad data for 5 different networks:  
Google, Meta, Microsoft, LinkedIn, Twitter

Without ForEach:

5 separate pipelines = messy, hard to maintain

With ForEach:

One loop, process all 5 automatically

### Hands On:

Drag "ForEach" onto canvas

Name: ForEach\_AdNetworks

Items (what to loop over):

```
@{json('["Google","Meta","Microsoft","LinkedIn","Twitter"]')}
```

OR **from** pipeline parameter:

```
@{pipeline().parameters.NetworkList}
```

IsSequential:

**false** = process all **in PARALLEL** (faster)

**true** = process ONE BY **ONE** (safer, use **when** order matters)

Batch Count: 3 (**if** parallel, how many at once)

→ Click pencil icon to **add** activities INSIDE ForEach:

Inside: Copy Activity

→ Source uses: `@{item()}` to **get** current network name

→ File: `@{concat(item(), '_ads_',  
pipeline().parameters.ProcessDate, '.csv')}`

Now ADF loops through all **5** networks automatically!

→ `@item()` gives you: **"Google"** first, **"Meta"** second, etc.

## 7.4 Lookup Activity — Read Config from Database

Real use **case**:

*Don't hardcode table names or settings in pipeline*

Store them **in** a **"control table"** **in** SQL

Lookup reads **from** control table → drives the pipeline

This **is** called **"Metadata-Driven Pipeline"** → Ansh teaches this **as**  
a **KEY** pattern

**Create control table:**

```
CREATE TABLE etl.PipelineControl (
    Id                INT IDENTITY PRIMARY KEY,
    SourceSystem      VARCHAR(50),
    SourceFile        VARCHAR(200),
    TargetTable       VARCHAR(100),
    IsActive          BIT DEFAULT 1,
    LoadOrder        INT
);

INSERT INTO etl.PipelineControl VALUES
    ('Google',      'google_ads_*.csv',      'dbo.GoogleAds',      1, 1),
    ('Meta',        'meta_ads_*.csv',        'dbo.MetaAds',        1, 2),
    ('Microsoft',   'microsoft_ads_*.csv',   'dbo.MicrosoftAds',   1, 3);
```

### Add Lookup Activity:

Drag "Lookup" onto canvas

Name: Lookup\_GetActiveSources

Source dataset: Create new → Azure SQL → select dataset

Query: `SELECT * FROM etl.PipelineControl WHERE IsActive = 1 ORDER BY LoadOrder`

First row only: ❌ (we want ALL rows, not just first)

Output:

`@activity('Lookup_GetActiveSources').output.value`  
→ Returns array of all rows

`@activity('Lookup_GetActiveSources').output.count`  
→ How many rows returned

### Pass to ForEach:

ForEach activity:

Items: @{activity('Lookup\_GetActiveSources').output.value}

Inside ForEach:

```
@item().SourceSystem → "Google"
@item().SourceFile   → "google_ads_*.csv"
@item().TargetTable  → "dbo.GoogleAds"
```

---

## 7.5 Wait Activity

Drag "Wait" onto canvas

Name: Wait\_5Minutes

Wait **time in** seconds: **300**

Use **case**:

After triggering an **external** process,  
wait **5** minutes before checking its status

---

## 7.6 Execute Pipeline Activity — Modular Design

Real projects have MANY pipelines.

*Don't put everything in ONE giant pipeline.*

Break **into** smaller pipelines, **call** them **from** a master.

```
pl_Master_Orchestrator      (the boss)
  └─ Execute → pl_IngestGoogleAds
  └─ Execute → pl_IngestMetaAds
```



```
|— Execute → pl_TransformData  
|— Execute → pl_LoadToSynapse
```

Master pipeline → runs them all → controls **order**

**Sub-pipelines** → focused **on** one job

## Hands On:

Drag **"Execute Pipeline"** activity

Name: Execute\_IngestGoogleAds

Invoked pipeline: pl\_IngestGoogleAds (**select from** dropdown)

Parameters (pass values to child pipeline):

ProcessDate: @{pipeline().parameters.ProcessDate}

Network: **"Google"**

Wait **on** completion:

☒ **true** = wait **for** child to finish before moving **on**

☐ **false** = fire **and forget** (parallel)

---

# **TOPIC 8 — Mapping Data Flows**

## **Visual Data Transformation (Ansh's Favourite Topic)**

---

### **What Is Mapping Data Flow — Ansh's Way**

Ansh explains it **like** this:

Without Data Flow:

Write Python/Spark code **to** clean data

Need **to** know PySpark, cluster management

Complex **for** beginners

**WITH** Mapping Data Flow:

DRAG **and** DROP transformation blocks

ADF writes the Spark code **for** you

Runs **on** managed Spark cluster

You click → Data transforms → DONE

*It's like Excel formulas but for BIG DATA.*

**Handles** millions **of** rows easily.

---

## Data Flow Transformations — One By One (Ansh Covers All)

|                   |                                              |
|-------------------|----------------------------------------------|
| SOURCE            | → <b>where</b> data comes <b>in</b>          |
| FILTER            | → remove rows you don't <i>want</i>          |
| <b>SELECT</b>     | → pick/rename/drop columns                   |
| DERIVED COLUMN    | → create <b>new</b> columns, modify existing |
| CAST              | → change data types                          |
| <b>JOIN</b>       | → combine two datasets                       |
| LOOKUP            | → enrich <b>from</b> a reference table       |
| <b>AGGREGATE</b>  | → <b>GROUP BY</b> , sum, count, avg          |
| CONDITIONAL SPLIT | → split stream <b>into</b> multiple outputs  |
| UNION             | → combine multiple streams                   |
| SORT              | → <b>order</b> rows                          |
| WINDOW            | → running total, rank, lag/lead              |
| SINK              | → <b>where</b> data goes out                 |

# Create Your First Data Flow — Full Hands On

## Scenario:

We have ad clicks CSV (raw, messy):

- Some rows have null AdCost
- Some rows have AdCost = -1 (invalid)
- Need to add column: IsHighValue (AdCost > 3.00)
- Need to add column: LoadDate = today
- Load clean data into SQL table

## Step 1 — Create Data Flow:

Author → Data Flows → + New Data Flow

Name: df\_CleanAdClicks

- Turn ON "Data Flow Debug" (top right toggle)
  - ← This starts a Spark cluster (3-4 min)
  - ← Keep TTL = 60 minutes so cluster stays warm

## Step 2 — Add Source:

Click "Add Source" on canvas

Source settings tab:

|              |                    |
|--------------|--------------------|
| Name:        | source_RawAdClicks |
| Source type: | Dataset            |
| Dataset:     | ds_AdClicks_CSV    |

Options:

|                     |                                                                        |
|---------------------|------------------------------------------------------------------------|
| Allow schema drift: | <input checked="" type="checkbox"/> (handles extra columns gracefully) |
| Validate schema:    | <input type="checkbox"/>                                               |

→ Click "Data Preview" tab → see your sample rows

### Step 3 — Add Filter (remove bad rows):

Click + on source\_RawAdClicks → Add Filter

Name: filter\_RemoveBadData

Filter on: `isNull(ad_id) == false()`  
          `&& isNull(user_id) == false()`  
          `&& ad_cost >= 0`

Translation: Keep only rows where:

ad\_id is NOT null  
user\_id is NOT null  
ad\_cost is 0 or positive

→ Data Preview: bad rows removed

### Step 4 — Add Derived Column (add new columns):

Click + on filter → Add Derived Column

Name: derive\_AddColumns

Add new columns:

Column: IsHighValue

Expression: `ad_cost > 3.00`  
(returns true/false)

Column: LoadDate

Expression: `currentDate()`  
(today's date)

Column: AdCostCategory

Expression: `iif(ad_cost > 5, 'Premium',`

```
iif(ad_cost > 3, 'High',  
    iif(ad_cost > 1, 'Medium', 'Low'))
```

Column: ClickedBit

Expression: toBoolean(clicked)

(convert string "true"/"false" to proper boolean)

→ Data Preview: 4 new columns added to each row

### Step 5 — Add Select (clean up columns):

Click + → Add Select

Name: select\_FinalColumns

Mode: Rule-based mapping

Keep these columns:

|                |                         |
|----------------|-------------------------|
| ad_id          | → rename to: AdId       |
| user_id        | → rename to: UserId     |
| campaign_id    | → rename to: CampaignId |
| ad_cost        | → rename to: AdCost     |
| timestamp      | → rename to: Timestamp  |
| IsHighValue    | → (keep as is)          |
| LoadDate       | → (keep as is)          |
| AdCostCategory | → (keep as is)          |
| ClickedBit     | → rename to: IsClicked  |

Drop: clicked (original string version)

→ Data Preview: clean final column set

### Step 6 — Add Sink (write to SQL):

Click + → **Add Sink**

Name: sink\_CleanAdClicks

Sink type: Dataset

Dataset: ds\_SQL\_AdClicks

Settings tab:

**Update method:** Allow **insert**

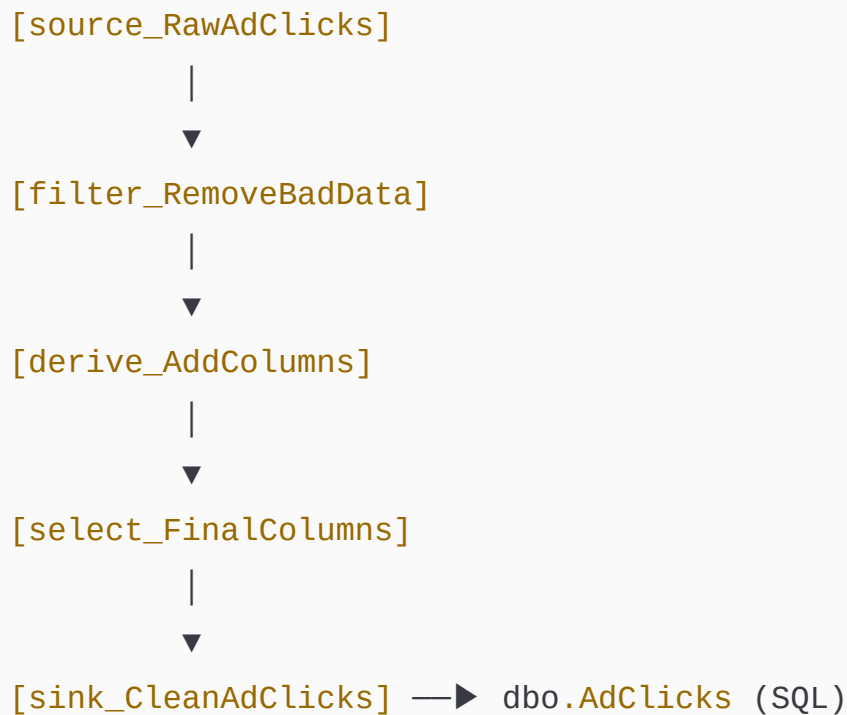
**Table action:** **None** (table already **exists**)

Mapping tab:

Auto mapping: ☒ (maps same-name columns automatically)

→ Verify mapping looks correct

## Step 7 — Final Data Flow Picture:



## Add Data Flow to Pipeline

Go back **to** your pipeline: `pl_CopyAdClicks_BlobToSQL`

Drag "Data Flow" activity **from** activities panel

Name: `Transform_CleanAdClicks`

Data flow: `df_CleanAdClicks` (**select from** dropdown)

**Connect** it AFTER your **Copy** activity

(**or** remove **Copy** and use Data Flow **as both copy** + transform)

Run Settings:

Compute type: General purpose

Core count: **8 + 8** (driver + worker)

→ Debug → watch it run **on** Spark

→ **Check SQL table**: **rows are** clean, **new** columns present 

## Join Two Streams in Data Flow — Ads + Campaign Details

Scenario:

`ad_clicks` has `CampaignId` but **not** Campaign Name

We need **to JOIN with** `dbo.Campaigns` **to get** Campaign Name

**Add second** Source:

Source **2**: `source_Campaigns`

Dataset: `ds_SQL_Campaigns` (points **to** `dbo.Campaigns`)

**Add Join** block:

```
Name: join_AddCampaignDetails
Left stream:  filter_RemoveBadData (ad clicks)
Right stream: source_Campaigns
Join type:    Left outer
              (keep all ad clicks even if campaign not found)
Join conditions:
  Left column: campaign_id
  Right column: CampaignId

Output: all ad click columns + CampaignName, Budget,
AdvertiserName
```

## Aggregate in Data Flow — Calculate KPIs

Add Aggregate block AFTER Join

Name: agg\_CalculateKPIs

Group By:

CampaignId  
CampaignName  
LoadDate

Aggregates:

```
TotalImpressions = count()
TotalClicks      = sum(iif(IsClicked, 1, 0))
TotalSpend       = round(sum(AdCost), 2)
CTR              = round(divide(sum(iif(IsClicked, 1, 0)),
count()) * 100, 4)
AvgCostPerClick  = iif(sum(iif(IsClicked, 1, 0)) > 0,
                      round(divide(sum(AdCost),
sum(iif(IsClicked, 1, 0))), 2),
```



```
0.0)  
HighValueCount = sum(iif(IsHighValue, 1, 0))
```

→ Add Sink: sink\_CampaignKPIs → dbo.CampaignKPIs table

## Conditional Split — Route Different Data

Scenario:

High value ads (> \$5) → VIP table (special monitoring)

Normal ads → regular table

Add Conditional Split AFTER derive\_AddColumns

Name: split\_ByValue

Output stream 1: highValue

Condition: ad\_cost > 5.0

Output stream 2: normalValue (default/else)

All remaining rows

→ Add Sink for highValue → dbo.VIPAds

→ Add Sink for normalValue → dbo.RegularAds

## TOPIC 9 — Triggers

### When Do Pipelines Run?

## 9.1 Schedule Trigger — Run at Fixed Time

The most common trigger.  
Like setting an alarm **clock**.

"Run this pipeline every day at 11 PM"  
"Run every Monday at 6 AM"  
"Run every 15 minutes"

### Create Schedule Trigger:

Author → Triggers → + New

**Name:** trigger\_DailyAdProcessing  
**Description:** Runs ad processing pipeline every night at **11** PM

**Type:** Schedule

**Start date:** 2026-01-01 11:00 PM UTC  
**Recurrence:** Every 1 Day  
**End:** Never (or set end date)

→ Click Next → **Pipeline:** pl\_CopyAdClicks\_BlobToSQL

**Pipeline Parameters:**

**ProcessDate:** @{formatDateTime(trigger().scheduledTime, 'yyyy-MM-dd')}

← **THIS IS KEY:** trigger().scheduledTime gives the SCHEDULED time  
not utcNow() because if pipeline runs late, we still want  
the  
correct date it was SUPPOSED to run for

→ Finish → Publish → Toggle to ON

## 9.2 Storage Event Trigger — Run When File Arrives

Most exciting **trigger for real-time** use cases.  
**Like** a doorbell – rings **when** something arrives.

"When any .csv file is uploaded to raw-data container  
→ immediately start processing pipeline"

**No** waiting **for** scheduled time.

File arrives **at** 3:47 PM → pipeline starts **at** ~3:49 PM.

### Create Event Trigger:

+ New Trigger → Storage Event

Name: trigger\_FileArrival\_AdClicks

Type: Storage events

Storage account: stadflabXXXX (select from dropdown)

Container name: /raw-data

Blob path begins with: /ad-clicks/

Blob path ends with: .csv

Event: Blob created ☒ (trigger on new file)

Blob deleted ☐

→ Next → Pipeline: pl\_CopyAdClicks\_BlobToSQL

Pipeline Parameters:

ProcessDate: @{formatDateTime(trigger().startTime, 'yyyy-MM-dd')}

→ Finish → Publish → Activate

**NOW:** Upload a CSV to raw-data/ad-clicks/ container

→ Within 2-3 minutes → pipeline runs automatically! 

## 9.3 Tumbling Window Trigger — Process Data Windows

Think of it as processing TIME WINDOWS.

"Process 1 hour of data at a time"

"Each run processes: 9AM-10AM, then 10AM-11AM, etc."

Windows don't overlap.

Windows run in ORDER.

If pipeline was down → BACKFILLS automatically.

Perfect for: hourly metrics, event-time processing

### Create Tumbling Window Trigger:

+ New Trigger → Tumbling Window

Name: trigger\_HourlyKPIs

Recurrence: Every 1 Hour

Start: 2026-01-01 00:00 UTC

Max concurrency: 3 (3 windows can run in parallel)

Retry: 2 (retry failed window up to 2 times)

Retry interval: 30 minutes

→ Pipeline Parameters:

WindowStart:

```
@{formatDateTime(trigger().outputs.windowStartTime, 'yyyy-MM-ddTHH:mm:ss')}
```

WindowEnd:    

```
@{formatDateTime(trigger().outputs.windowEndTime, 'yyyy-MM-ddTHH:mm:ss')}
```

In your pipeline, use WHERE clause:

```
WHERE timestamp >= '@{pipeline().parameters.WindowStart}'  
AND   timestamp <  '@{pipeline().parameters.WindowEnd}'
```

This ensures EXACTLY the right data for each window.

If ADF was down for 3 hours → triggers 3 missed windows → backtracks automatically!

---

## TOPIC 10 — Medallion Architecture

### Ansh's Signature Project Pattern — Bronze, Silver, Gold

---

#### What Is Medallion Architecture

Ansh teaches all end-to-end projects using this pattern.  
It's the *INDUSTRY STANDARD* for data lakes.

Think of it like refining gold:

BRONZE (Raw)

→ Exact copy of source data

- No transformations
- Append only (keep history)
- "This is what we received"

#### SILVER (Cleaned)

- Cleaned, validated, standardized
- Deduplication done
- Bad rows removed
- "This is trustworthy data"

#### GOLD (Business Ready)

- Aggregated, business logic applied
- Ready for reports and ML models
- "This is what business users see"

---

## Folder Structure in Data Lake (ADLS Gen2)

Azure Data Lake Gen2: datalake-ads

containers:

bronze/

ad-clicks/

2026/02/20/

ad\_clicks\_google.csv (exact copy of source)

ad\_clicks\_meta.csv

campaigns/

2026/02/20/

campaigns.csv

silver/

ad-clicks/

2026/02/20/

```
    ad_clicks_cleaned.parquet    (cleaned, validated)
campaigns/
    2026/02/20/
    campaigns_cleaned.parquet

gold/
    kpis/
    2026/02/20/
    campaign_kpis.parquet    (aggregated KPIs)
ml-features/
    2026/02/20/
    training_features.parquet
```

## Build Medallion Pipeline — Step by Step

### Pipeline 1: pl\_Bronze\_IngestRawData

**Purpose:** Copy raw files from source → Bronze layer  
Do NOT transform, just preserve

**Activities:**

1. CopyToBronze\_GoogleAds

**Source:** REST API (Google Ads API) OR Blob

**Sink:** ADLS Gen2 Bronze layer

**Sink path:** bronze/ad-

clicks/@{formatDateTime(utcNow(), 'yyyy/MM/dd')}/google\_ads.csv

**Write behavior:** Add dynamic file name

2. CopyToBronze\_MetaAds

**Source:** Meta Ads source

**Sink:** bronze/ad-clicks/.../meta\_ads.csv

### 3. CopyToBronze\_Campaigns

**Source:** SQL DB campaigns table

**Sink:** bronze/campaigns/.../campaigns.csv

- Bronze layer has EXACT COPY of raw data
- Always there for audit/reprocessing

## Pipeline 2: pl\_Silver\_CleanData (Data Flow)

**Purpose:** Clean bronze → write to Silver as Parquet

**Data Flow:** df\_BronzeToSilver

**Source:** Read from Bronze (Parquet or CSV)

**source\_BronzeAdClicks**

**Path:** bronze/ad-

clicks/@{pipeline().parameters.ProcessDate}/\*.csv

**Transformations:**

filter → remove nulls, negative costs

derive → add LoadDate, IsHighValue, AdCostCategory

cast → correct data types

select → final columns

**Sink:** Write to Silver as Parquet (much faster than CSV)

**Path:** silver/ad-clicks/@{pipeline().parameters.ProcessDate}/

**Format:** Parquet

**Compression:** snappy

**Why Parquet?**

- 5-10x faster to read than CSV
- Compressed (smaller file size)
- Column-based (read only columns you need)
- Industry standard for data lakes



### Pipeline 3: pl\_Gold\_CalculateKPIs (Data Flow)

**Purpose:** Transform Silver → Gold business metrics

**Data Flow:** df\_SilverToGold

**Source 1:** Silver ad clicks Parquet

**Source 2:** Silver campaigns Parquet

**Join:** ad clicks JOIN campaigns ON CampaignId

**Aggregate:**

**Group By:** CampaignId, CampaignName, ProcessDate

**Metrics:**

```
Impressions    = count()
Clicks         = sum(iif(IsClicked, 1, 0))
TotalSpend     = sum(AdCost)
CTR            = divide(clicks, impressions) * 100
AvgCPC         = divide(TotalSpend, Clicks)
```

**Sink 1:** Gold Parquet (for Synapse and ML)

**Path:** gold/kpis/@{pipeline().parameters.ProcessDate}/

**Sink 2:** Azure SQL (for Power BI)

**Table:** dbo.DailyCampaignKPIs

**Write:** Upsert on CampaignId + ProcessDate

### Master Orchestrator:

**Pipeline:** pl\_Master\_DailyProcess

**Parameter:** ProcessDate = @{formatDateTime(utcNow(), 'yyyy-MM-dd')}

**Activities in ORDER:**

1. Execute\_Bronze
  - pl\_Bronze\_IngestRawData
  - Wait: true
  - On Success → go to step 2
2. Execute\_Silver
  - pl\_Silver\_CleanData
  - Wait: true
  - On Success → go to step 3
3. Execute\_Gold
  - pl\_Gold\_CalculateKPIs
  - Wait: true
  - On Success → go to step 4
4. Notify\_Success (Web Activity)
  - Body: "✅ Daily pipeline done. Date: @{{pipeline().parameters.ProcessDate}}"

On any failure:

5. Notify\_Failure (Web Activity)
  - Body: "❌ Pipeline FAILED: @{{pipeline().Pipeline}} at @{{utcNow()}}"

---

## **TOPIC 11 — Incremental Loading**

**Load ONLY New Data (Ansh Teaches This as Critical Pattern)**

---

## The Problem With Full Load Every Time

Full Load every day:

Day 1: Table has 100,000 rows → copy all 100,000

Day 2: Table has 110,000 rows → copy ALL 110,000 again!

Day 3: Table has 125,000 rows → copy ALL 125,000 again!

After 1 year:

Table has 36 million rows

Copying 36 million rows every night = SLOW + EXPENSIVE

INCREMENTAL LOAD:

Day 1: Copy 100,000 rows (all new)

Day 2: Copy only 10,000 NEW rows

Day 3: Copy only 15,000 NEW rows

Much faster. Much cheaper. 

---

## Watermark Pattern — Full Hands On

### Step 1: Create Watermark Table:

```
-- In Azure SQL
CREATE TABLE etl.WatermarkControl (
    TableName          VARCHAR(100) PRIMARY KEY,
    LastWatermarkVal DATETIME2
);

INSERT INTO etl.WatermarkControl VALUES
('dbo.AdClicks', '1900-01-01 00:00:00');
```

### Step 2: Create Pipeline:

Pipeline: `pl_Incremental_AdClicks`

Activities:

Activity 1: `Lookup_OldWatermark`

Type: `Lookup`

Dataset: `ds_SQL_WatermarkControl` (SQL dataset)

Query: `SELECT LastWatermarkVal FROM etl.WatermarkControl  
WHERE TableName = 'dbo.AdClicks'`

First row only: ☒

Activity 2: `Lookup_NewWatermark` (parallel with Activity 1)

Type: `Lookup`

Dataset: `ds_SQL_AdClicks` (source table)

Query: `SELECT MAX(Timestamp) AS NewWatermark FROM dbo.AdClicks`

First row only: ☒

Activity 3: `CopyIncremental_AdClicks`

Depends on: Activity 1 AND Activity 2 (both must succeed)

Type: `Copy`

Source dataset: `ds_SQL_AdClicks`

Source query:

`SELECT * FROM dbo.AdClicks  
WHERE Timestamp >`

`'@{activity('Lookup_OldWatermark').output.firstRow.LastWatermarkVal}  
AND Timestamp <=`

`'@{activity('Lookup_NewWatermark').output.firstRow.NewWatermark}'`

Sink: `ds_SQL_AdsTarget` (destination table)

Write: `Insert`

Activity 4: `SP_UpdateWatermark`

Depends on: Activity 3 Success

Type: `Stored Procedure`

SQL stored procedure:

```
CREATE PROCEDURE usp_UpdateWatermark
    @TableName NVARCHAR(100),
    @NewWatermark DATETIME2
AS
UPDATE etl.WatermarkControl
SET LastWatermarkVal = @NewWatermark
WHERE TableName = @TableName;
```

Parameters:

TableName: "dbo.AdClicks"

NewWatermark:

@{activity('Lookup\_NewWatermark').output.firstRow.NewWatermark}

**Run it:**

First run:

Old watermark: 1900-01-01 (all rows are newer → copies all)

New watermark: 2026-02-20 10:00:00

Updates control table to 2026-02-20 10:00:00

Second run (an hour later):

Old watermark: 2026-02-20 10:00:00

New watermark: 2026-02-20 11:00:00

Copies ONLY rows between 10AM and 11AM ← incremental!



## TOPIC 12 — SCD Type 1 and Type

2

# Ansh Teaches This for Data Warehousing

## What Are Slowly Changing Dimensions

In a data warehouse, dimension tables (Campaigns, Advertisers) change slowly over time. Not every day, but occasionally.

Question: When data changes, what do you do?

SCD Type 1: Overwrite (no history kept)

Campaign budget changed from \$1000 to \$1500

- Just update the record
- Old value GONE → no history
- Use when history doesn't matter

SCD Type 2: Versioning (history preserved)

Campaign budget changed from \$1000 to \$1500

- Mark old record as INACTIVE
- Insert new record as ACTIVE
- Both records exist → full history
- Use when history is important (auditing, reporting)

## SCD Type 1 — Upsert Pattern in ADF Data Flow

In Data Flow — Alter Row transformation:

Add Alter Row BEFORE Sink:

Name: alter\_UpsertLogic

Conditions:

→ Upsert if: `true()`  
(mark `all rows for` upsert)

Sink settings:

`Update method`: Allow upsert

Key columns: CampaignId (`unique` identifier)

`Result`:

`New` campaigns → INSERTED

Changed campaigns → UPDATED (`old values` overwritten)

→ History LOST (SCD Type `1`)

## SCD Type 2 — Full Implementation in ADF

**Destination table structure:**

```
CREATE TABLE dbo.DimCampaigns_SCD2 (  
    SurrogateKey      INT IDENTITY PRIMARY KEY,  
    CampaignId        VARCHAR(10),          -- natural key  
    CampaignName       VARCHAR(100),  
    Budget            DECIMAL(10,2),  
    Status            VARCHAR(20),  
    -- SCD2 tracking columns:  
    StartDate         DATETIME2,            -- when this version became  
    active  
    EndDate           DATETIME2,            -- when this version was  
    replaced (NULL = still active)  
    IsCurrent         BIT DEFAULT 1         -- 1 = current version  
);
```

**Data Flow: df\_SCD2\_Campaigns:**

Source 1: source\_NewCampaigns

→ Incoming data from API/file

Source 2: source\_ExistingCampaigns

→ `SELECT * FROM dbo.DimCampaigns_SCD2 WHERE IsCurrent = 1`

→ Only active records

Step 1: Lookup to find which rows changed

→ Left join new with existing on CampaignId

Step 2: Derived Column to detect changes

```
HasChanged = !equals(newData.Budget, existingData.Budget)
              || !equals(newData.CampaignName,
existingData.CampaignName)
```

Step 3: Conditional Split

Stream 1: newRecords → CampaignId NOT in existing (brand new)

Stream 2: changedRows → CampaignId in existing AND HasChanged  
= true

Stream 3: unchanged → HasChanged = false (skip these)

Step 4: For changedRows → TWO sinks:

Sink A: Update old record (set IsCurrent = 0, EndDate = today)

Alter Row: Update if: true()

Update: IsCurrent = 0, EndDate = currentDate()

Key: SurrogateKey

Sink B: Insert new version

StartDate = currentDate()

EndDate = NULL

IsCurrent = 1

Step 5: For newRecords → Insert



```
StartDate = currentDate()  
IsCurrent = 1
```

## **TOPIC 13 — Monitoring and Debugging**

**Ansh Says: "Know How to Debug = Senior Engineer"**

### **Monitor Hub — Your Eyes on Everything**

ADF Studio → Monitor (left sidebar)

Three main views:

#### 1. Pipeline Runs

Shows ALL pipeline executions

Columns: Pipeline Name, Status, Start Time, Duration, Trigger

Status icons:

-  Succeeded → all good
-  Failed → something broke
-  In Progress → still running
-  Queued → waiting for capacity
-  Cancelled → someone stopped it

#### 2. Activity Runs

Click on a pipeline run → see each activity

Which activity failed?

How long did each activity take?

How many rows were copied?

### 3. Trigger Runs

Shows when triggers fired

Did the 11PM trigger fire correctly?

## Debug a Failed Pipeline — Step By Step

Scenario: "Pipeline failed" alert came in. What do you do?

Step 1: Monitor → Pipeline Runs → Find failed run (red X)

Step 2: Click the pipeline name

→ See activity list

→ Failed activity shown in red

Step 3: Click glasses icon next to failed activity

→ Error tab:

ErrorCode: "2200" (user error)

Message: "Cannot insert NULL into column 'AdId'"

← Now you know EXACTLY what went wrong

Step 4: Click INPUT tab

→ See what parameters went into this activity

→ Verify ProcessDate, file paths, etc. are correct

Step 5: Click OUTPUT tab

→ rowsRead: 5000

→ rowsCopied: 0

→ rowsSkipped: 5000

→ errorFilePath: errors/run-XXX/incompatible-rows.csv

← Check that error file for the actual bad rows

### Step 6: Fix the issue

- Maybe source file had empty AdId column
- Add filter **in** data flow **to skip** null AdId rows
- **Or** add **NOT NULL** handling **in** derived column

### Step 7: Rerun **from** failure

- Click "Rerun" → "From Failed Activity"
- Only failed + downstream activities run
- Previously successful activities SKIPPED
- Saves time **and** cost

## Set Up Alerts — Never Miss a Failure

Monitor → Alerts → + New Alert Rule

### Alert 1: Pipeline Failure

Signal: Failed Pipeline Runs

Condition: Count  $\geq$  1

Evaluation: Every 5 minutes

Lookback: Last 5 minutes

Severity: Error (Sev 2)

Action Group: + Create New

Email: data-team@company.com

SMS: your phone number

Teams: webhook URL

### Alert 2: Long Running Pipeline

Signal: PipelineSucceededRuns

Condition: Duration > 3600 seconds (1 hour)

→ Alert if pipeline takes longer than expected

→ Might indicate performance degradation

→ Save and Enable both alerts

→ Now you'll get notified of any issues automatically

## Pipeline Run History — Checking Performance Over Time

Monitor → Pipeline Runs

Filter by:

Pipeline name: pl\_CopyAdClicks\_BlobToSQL

Status: All

Date range: Last 30 days

Sort by Duration (descending)

→ Find the slowest runs

→ Identify patterns (Monday always slow? End of month?)

→ Investigate and optimize

Export to CSV for detailed analysis

## TOPIC 14 — Git Integration and CI/CD

**Professional Development Practice (Ansh Covers This at End)**

## Why Git Integration

Without Git:

- You change pipeline → immediately live
- Mistake → breaks production
- No version history
- Can't *work with team*

With Git:

- Changes saved **to** git branch (**not** published yet)
- Team reviews via Pull Request
- Only approved changes go **to** production
- Full history **of** every change
- Work together without conflicts

## Set Up Git Integration

Manage → Git Configuration → Set Up Code Repository

Repository type: Azure DevOps Git (or GitHub)

Azure DevOps:

Azure DevOps account: `dev.azure.com/your-org`

Project name: `AdsDataPlatform`

Repository name: `adf-pipelines`

Collaboration branch: `main` ← where completed features go

Publish branch: `adf_publish` ← ADF auto-creates this

Root folder: `/`

→ Apply

Now you see branch name in top bar: main ▼

How it works:

- Create feature branch: feature/add-cleanup-pipeline
- Make changes in ADF Studio
- Changes auto-save to feature branch (not published)
- Create PR to main branch
- Team reviews → approves → merge
- In ADF: switch to main → click "Publish"
- Publishes to adf\_publish branch
- CI/CD pipeline deploys to test and production

---

## Azure DevOps Pipeline for ADF Deployment

```
# azure-pipelines-adf.yml

trigger:
  branches:
    include:
      - adf_publish # when ADF publishes → deploy

stages:

# — Deploy to TEST —————
- stage: DeployTest
  jobs:
    - job: Deploy
      steps:
        - checkout: self

# Stop triggers before deploy (prevents runs during deploy)
- task: AzurePowerShell@5
```

```

    displayName: 'Stop Triggers'
    inputs:
      azureSubscription: 'AzureServiceConnection'
      ScriptType: InlineScript
      Inline: |
        $triggers = Get-AzDataFactoryV2Trigger -
ResourceGroupName "rg-ads-test" -DataFactoryName "adf-ads-test"
        foreach ($trigger in $triggers) {
          Stop-AzDataFactoryV2Trigger -ResourceGroupName "rg-
ads-test" -DataFactoryName "adf-ads-test" -Name $trigger.Name -
Force
        }

# Deploy ARM template to test ADF
- task: AzureResourceManagerTemplateDeployment@3
  displayName: 'Deploy to Test ADF'
  inputs:
    azureResourceManagerConnection: 'AzureServiceConnection'
    resourceGroupName: 'rg-ads-test'
    location: 'East US'
    templateLocation: 'Linked artifact'
    csmFile:
'$(Build.SourcesDirectory)/ARMTemplateForFactory.json'
    csmParametersFile:
'$(Build.SourcesDirectory)/config/test-params.json'
    deploymentMode: Incremental

# Restart triggers after deploy
- task: AzurePowerShell@5
  displayName: 'Start Triggers'
  inputs:
    azureSubscription: 'AzureServiceConnection'
    ScriptType: InlineScript
    Inline: |
      Start-AzDataFactoryV2Trigger -ResourceGroupName "rg-

```

```
ads-test" -DataFactoryName "adf-ads-test" -Name  
"trigger_DailyAdProcessing"
```

```
# — Deploy to PRODUCTION —————
```

```
- stage: DeployProduction  
  dependsOn: DeployTest  
  condition: succeeded()  
  jobs:  
    - deployment: DeployProd  
      environment: 'Production' # requires manual approval  
      strategy:  
        runOnce:  
          deploy:  
            steps:  
              - task: AzureResourceManagerTemplateDeployment@3  
                inputs:  
                  resourceGroupName: 'rg-ads-prod'  
                  csmParametersFile: 'config/prod-params.json'
```

---

## TOPIC 15 — Complete End-to-End Project

### Ansh's Style — Real Project From Scratch

---

#### Project: Microsoft Ads Daily Analytics Pipeline

What we build:



#### SOURCE DATA:

- CSV files in Blob (ad clicks, impressions)
- Azure SQL (campaign configurations)

#### PIPELINE:

Bronze: Ingest raw → Data Lake Bronze layer

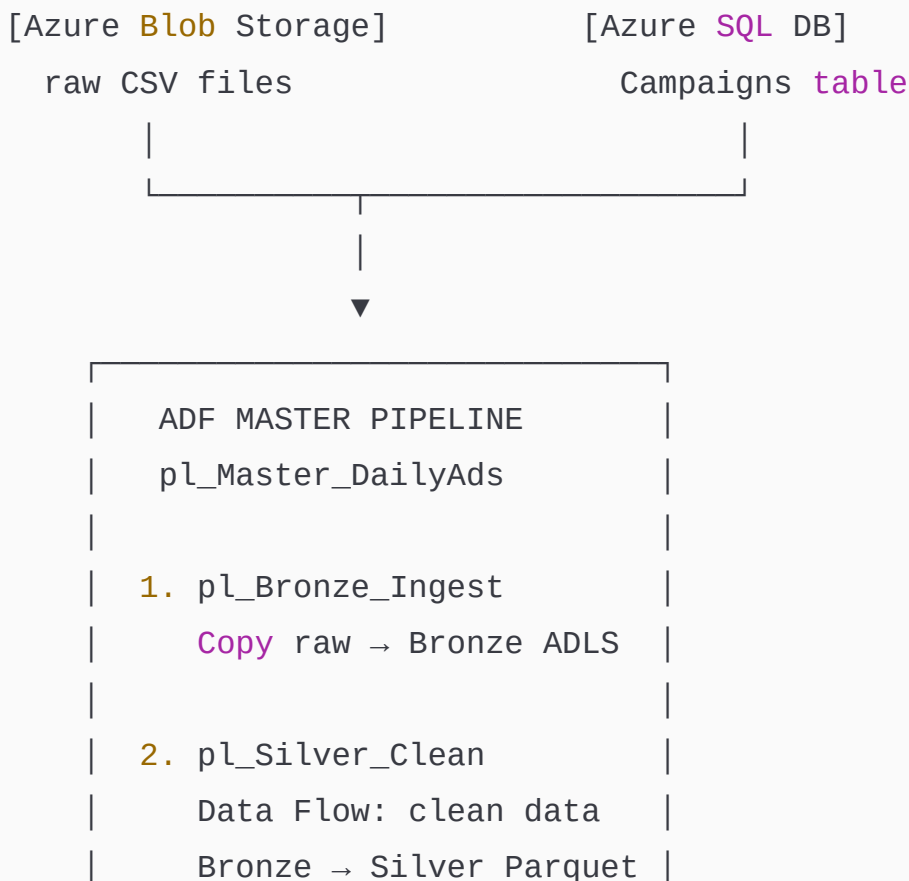
Silver: Clean and validate → Data Lake Silver layer (Parquet)

Gold: Calculate KPIs → SQL + Data Lake Gold layer

#### SCHEDULE:

- Runs every day at 11 PM
- Processes that day's data
- Sends Slack/Teams notification on completion

## Final Architecture — All Connected



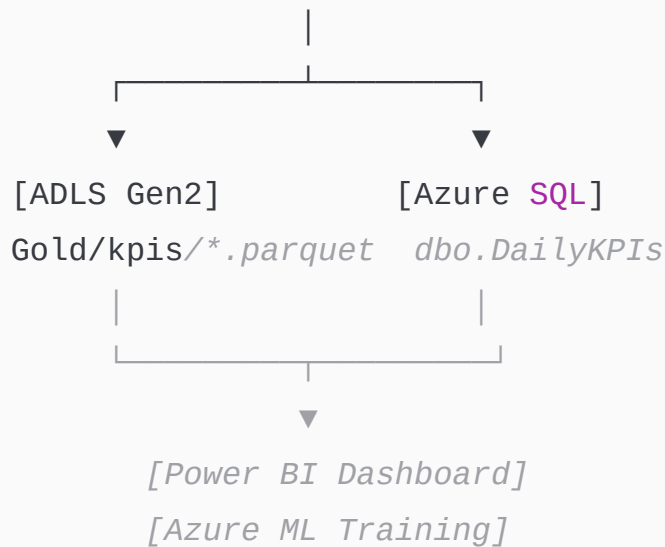
3. pl\_Gold\_KPIs

Data Flow: calc KPIs

Silver → Gold + SQL

4. pl\_Notify\_Teams

Send completion message



TRIGGER: Schedule → Every day 11 PM UTC

## Step-by-Step Build Order

Phase 1 – Infrastructure (30 min):

- ☐ Create Resource Group
- ☐ Create Storage Account + containers (raw, bronze, silver, gold)
- ☐ Create Azure Data Lake Gen2
- ☐ Create Azure SQL DB + tables
- ☐ Create Azure Data Factory

Phase 2 – Linked Services (15 min):

- ❑ ls\_BlobStorage\_Raw
- ❑ ls\_ADLS\_DataLake
- ❑ ls\_AzureSQL\_AdsDB

#### Phase 3 – Datasets (20 min):

- ❑ ds\_AdClicks\_CSV (blob, parameterized)
- ❑ ds\_Campaigns\_SQL (SQL table)
- ❑ ds\_Bronze\_AdClicks (ADLS bronze layer)
- ❑ ds\_Silver\_AdClicks (ADLS silver, Parquet)
- ❑ ds\_Gold\_KPIs (ADLS gold, Parquet)
- ❑ ds\_SQL\_DailyKPIs (SQL table)

#### Phase 4 – Bronze Pipeline (20 min):

- ❑ pl\_Bronze\_Ingest
- ❑ Copy CSV → Bronze ADLS
- ❑ Copy Campaigns → Bronze ADLS
- ❑ Test with Debug

#### Phase 5 – Silver Data Flow (45 min):

- ❑ df\_BronzeToSilver
- ❑ Source → Filter → Derive → Cast → Select → Sink
- ❑ Debug each step
- ❑ pl\_Silver\_Clean wraps data flow

#### Phase 6 – Gold Data Flow (45 min):

- ❑ df\_SilverToGold
- ❑ Source1 + Source2 → Join → Aggregate → Sink
- ❑ Write to both Parquet and SQL

#### Phase 7 – Master Pipeline (20 min):

- ❑ pl\_Master\_DailyAds
- ❑ Execute Bronze → Execute Silver → Execute Gold
- ❑ Add notifications

#### Phase 8 – Trigger (10 min):

- ☐ Schedule trigger at 11 PM
- ☐ Publish everything

Phase 9 — Test End to End (30 min):

- ☐ Upload test CSV to Blob
- ☐ Run master pipeline manually (Debug)
- ☐ Check Bronze → Silver → Gold data
- ☐ Verify SQL table populated
- ☐ Check monitoring dashboard

## Interview Q&A — Ansh Lamba Style

### Q: What is Azure Data Factory in simple words?

*ADF is Microsoft's cloud tool for moving and transforming data. It's like a smart conveyor belt — picks data from any source, cleans and transforms it, and delivers it to any destination automatically on schedule. No server management needed.*

### Q: What is Linked Service vs Dataset?

*Linked Service is the CONNECTION to a data store — like a phone contact with the address. Dataset is the specific DATA LOCATION and FORMAT — like saying which file or table inside that store. Linked Service answers HOW to connect, Dataset answers WHERE and WHAT.*

### Q: What is Integration Runtime?

*The compute engine that actually executes the work. Azure IR is serverless and managed by Microsoft for cloud-to-cloud transfers. Self-Hosted IR is installed on your own server for accessing on-premise data behind firewall.*

### Q: What is Mapping Data Flow?

*Visual transformation tool built on Apache Spark. You drag transformation blocks on a canvas. ADF generates Spark code and runs it on managed cluster. Handles millions of rows. No coding needed.*

### **Q: What is Medallion Architecture?**

*Three-layer data organization: Bronze (raw exact copy of source), Silver (cleaned and validated), Gold (aggregated business-ready data). Each layer improves data quality. Industry standard for data lakes.*

### **Q: What is Incremental Load and Watermark Pattern?**

*Instead of copying all data every run, only copy NEW data since last run. Watermark = a timestamp stored in control table tracking last successful load. Next run copies only rows newer than the watermark.*

### **Q: What is SCD Type 2?**

*Slowly Changing Dimension Type 2 — tracks history of changes. When data changes, mark old record as inactive (set IsCurrent=0, add EndDate), insert new record as active (IsCurrent=1). Both records preserved. Used for audit trails.*

### **Q: How do you handle pipeline failure?**

*Connect failure paths from activities to notification activities. Set up alerts in Monitor hub for automatic email/SMS notifications. Use retry settings on activities. Check Monitor hub for full error details. Use Rerun from Failure to restart only failed activity.*

## **Complete Topics Covered — Ansh Lamba Style**

-  Topic 1 — What Is ADF and Why It Exists
-  Topic 2 — Create ADF Instance (hands-on)
-  Topic 3 — Linked Services (create, parameterize, Key Vault)

- ✓ Topic 4 – Datasets (**create**, parameterize, reuse)
- ✓ Topic 5 – Integration Runtime (Azure IR, Self-Hosted IR)
- ✓ Topic 6 – **Copy** Activity (**full** settings, fault tolerance)
- ✓ Topic 7 – Parameters **and** Variables (expressions, **system** vars)
- ✓ Topic 8 – Control Flow Activities
  - **Get** Metadata, If **Condition**, ForEach
  - Lookup, Wait, **Execute** Pipeline
- ✓ Topic 9 – Mapping Data Flows (**all** transformations)
  - Source, **Filter**, Derived **Column**, **Select**
  - **Join**, Lookup, Aggregate, Conditional Split, Sink
- ✓ Topic 10 – Triggers (Schedule, Event, Tumbling **Window**)
- ✓ Topic 11 – Medallion Architecture (Bronze/Silver/Gold)
- ✓ Topic 12 – Incremental Loading (Watermark **Pattern**)
- ✓ Topic 13 – SCD Type 1 **and** Type 2
- ✓ Topic 14 – Monitoring **and** Debugging
- ✓ Topic 15 – Git Integration **and** CI/CD
- ✓ Topic 16 – Complete **End-to-End** Project (Microsoft Ads)
- ✓ Topic 17 – Interview Questions **and** Answers

---

Reply "**next**" and we go into **Azure Databricks with ADF, Synapse Analytics**, or **Azure Data Lake Gen2 deep dive** — all Ansh Lamba style, simple words, hands-on every step. 🚀

23/02/2026, 21:48:52